

Setup

```
git clone https://github.com/majirky/spacelab_ss26
cd spacelab_ss26

python -m venv .venv
source .venv/bin/activate      # mac / linux
.venv\Scripts\Activate.ps1    # windows powershell

python start/analysis.py      # očakava sa výstup ....
```

**Ak chcete uploadovať svoj package na Test Python repozitár (záver workshopu), je potrebné mať TOTP (Time-based One Time Password) aplikáciu, napr. Microsoft Authenticator*



Ako si vytvoriť vlastný Python Package

SPACE::LAB summer school 26
workshop

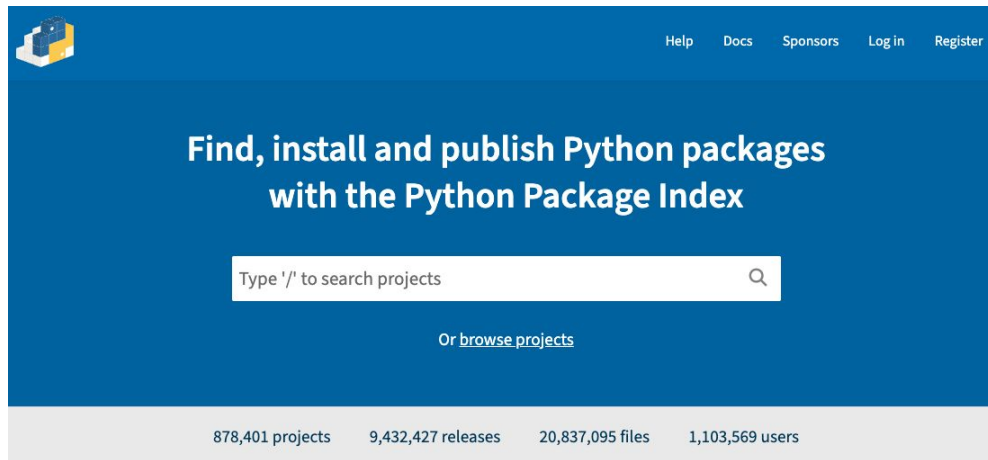
```
pip install pandas
```

Čo Vlastne pip install Robí?

Jeden príkaz potichu spraví päť vecí:

- **Nájde** balík medzi státisícami na PyPI
- **Vyrieši závislosti**, nájde kompatibilné verzie balíka a jeho závislostí
- **Vyberie správnu distribúciu** podľa OS, verzie Pythonu a architektúry CPU
- **Zvolí vhodný spôsob inštalácie**, zdrojový kód alebo predkompilovanú binárku (*wheel*)
- **Nainštaluje balík do prostredia a sprístupní ho Pythonu**; verzie umožňujú inštaláciu reprodukovateľ alebo balík odstrániť

PyPI udržiavajú dobrovoľníci vo voľnom čase



Prečo Je To Dôležité?

- **Reprodukovateľnosť**

Ak si recenzent dokáže váš kód jednoducho spustiť, má väčšiu dôveru vo výsledky výskumu. Vyššie šance na publikovanie.

- **Adopcia**

Ľudia častejšie používajú nástroje, ktoré sa dajú ľahko nainštalovať a integrovať do vlastnej práce.

- **Ty o 6 mesiacov**

Najčastejším používateľom tvojho kódu budeš pravdepodobne ty sám.

Nie všetko ale musí byť package.

Základne Pojmy Python Packagingu

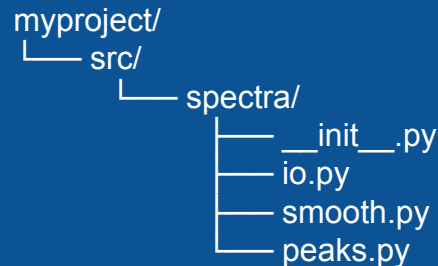
Modul (Module)	Jeden súbor <code>.py</code> obsahujúci Python kód.
Balík (Package)	Adresár obsahujúci jeden alebo viac modulov, prípadne ďalšie podbalíky.
Distribúcia (Distribution)	Balík určený na inštaláciu. Jeho názov sa môže líšiť od názvu používaného pri importe (scikit-learn/scikit).
Wheel (.whl)	Predpripravený balík obsahujúci kód a metadáta. Umožňuje rýchlu inštaláciu bez kompilácie.
Virtuálne prostredie (.venv)	Izolované Python prostredie s vlastnými knižnicami a závislosťami pre konkrétny projekt.
PyPI	Centrálné úložisko Python balíkov, z ktorého inštaluje <code>pip</code> .
pip	Nástroj na inštaláciu a správu Python balíkov.
pyproject.toml	Konfiguračný súbor definujúci metadáta projektu a spôsob jeho zostavenia.

Cvičenie 1 | Dajte Kódu Štruktúru

(10 min.)

Prerobte obsah súboru **analysis.py** do troch modulov:

- `io.py`
- `smooth.py`
- `peaks.py`



Pravidlá

1. **Žiadne cesty k súborom vo funkciách**
 - Cestu k súboru `.csv` odovzdáva volajúci kód ako vstupný parameter.
2. **Žiadne `print()` vo funkciách**
 - Funkcie vracajú hodnoty, nevypisujú ich na obrazovku.
3. **Pri importe sa nesmie nič spustiť**
 - Import modulu nesmie mať žiadne viditeľné vedľajšie efekty (nevykonávajú výpočty ani výpisy).
4. **Moduly umiestnite do adresára `src/`**

* nezabudnite na konštanty `FILE`, `WINDOW`, `TRESHOLD`. To by mal byť nastaviteľný vstupný parameter prislúchajúcej funkcie.

Na Niečo Sme Zabudli...

pyproject.toml, README, LICENSE

README.md

```
# spectra Find peaks in 1-D spectra.
```

```
## Install
```

```
```bash  
pip install spectra-yourname
```
```

```
## Use it
```

```
```python  
from spectra import load_csv, find_peaks
x, y = load_csv("data.csv")
print(find_peaks(x, y, threshold=4))
```
```

LICENSE

MIT

Rob si s tým, čo chceš, len zachovaj moje meno.

Najkratšia a najvoľnejšia licencia.
Predvolená voľba v rámci vedeckého Pythonu. Iní ju môžu dokonca predávať – musia však zachovať uvedenie autora.

Apache-2.0

MIT + výslovné udelenie patentových práv

Podobná ako MIT, ale navyše poskytuje patentovú ochranu a vyžaduje oznámenie o zmenách. Vhodná najmä tam, kde môžu byť relevantné patenty.

GPL-3.0

Zdieľaj za rovnakých podmienok – copyleft

Každý, kto distribuuje upravenú verziu, ju musí tiež zverejniť ako open-source. Zachováva odvodené diela slobodné; niektoré firmy sa jej preto vyhýbajú.

choosealicense.com

pyproject.toml

Je to deklaratívny súbor, ktorý odpovie na štyri základné otázky

Ako sa vybudujem?

[build-system]

nástroj na budovanie .whl

Čo som zač?

[project]

názov, verzia, popis, Python verzia k behu

Čo potrebujem?

dependencies

balíčky ktoré musí pip nainštalovať

Aké nástroje mám nastavené?

[tool.*]

nastavenia pre lintery, pytest, ...

Anatómia Minimálneho pyproject.toml

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "spectra-yourname"
version = "0.1.0"
description = "Find peaks in 1-D spectra."
readme = "README.md"
requires-python = ">=3.9"
license = "MIT"
dependencies = ["rich>=13.0"]

[project.scripts]
spectra-find = "spectra.cli:main"

[tool.hatch.build.targets.wheel]
packages = ["src/spectra"]
```

the build backend

nástroj, ktorý vytvára wheel balíček, takmer nikdy ho netreba meniť

name + version

tvoj názov na PyPI; verzia sa musí pri každom vydaní zvýšiť

requires-python

pip odmietne nainštalovať balík na starších verziách Pythonu

dependencies

nainštalujú sa automaticky

[project.scripts]

funkcie je tak možné volať z terminálu

packages

miesto, kde sa tvoj kód nachádza (src layout)

Cvičenie 2 | Metadáta

(5 min.)

V **myproject/** vytvorte dokumenty, ktoré popisujú Váš projekt:

- pyproject.toml (stačí minimálny príklad)
- README.md
- LICENSE.md

Buildovanie

```
(venv.) majirky@pc-adam spacelab_ss26 % pip install build      # nainštalujeme knižnicu na budovanie
(venv.) majirky@pc-adam spacelab_ss26 % python -m build      # spustíme knižnicu ako python module

(venv.) majirky@pc-adam spacelab_ss26 % ls dist/              # mali by sme dostať 2 súbory

(venv.) majirky@pc-adam spacelab_ss26 % python -m zipfile -l dist/*.whl  # čo sa nachádza v .whl súbore?
```

Cvičenie 3 | Použite Knižnicu

(5-10 min.)

Čo ste si vyrobili, to si aj vyskúšajte:

- Vytvorte si nový projekt (priechinok)
- Vytvorte si v ňom nové virtuálne prostredie pythonu .venv
- Nainštalujte si svoju knižnicu: `pip install "celá/cesta/k/suboru.whl"` (všimnite si úvodzovky)
- Vytvorte si `main.py`, naimportujte knižnicu (`import spectra`)
- Použite funkcie, alebo si vyprintujte verziu: `print(spectra.__version__)`

Vývojarská Empatia

Dva zvyky pre ktoré ťa budú mať ostatní radi:

```
def find_peaks(  
    x: list[float],  
    y: list[float]  
    ) -> list[tuple(float)]:  
    """  
    popis funkcie  
    """  
  
    pass  
    return [(0.0, 0.0)]
```

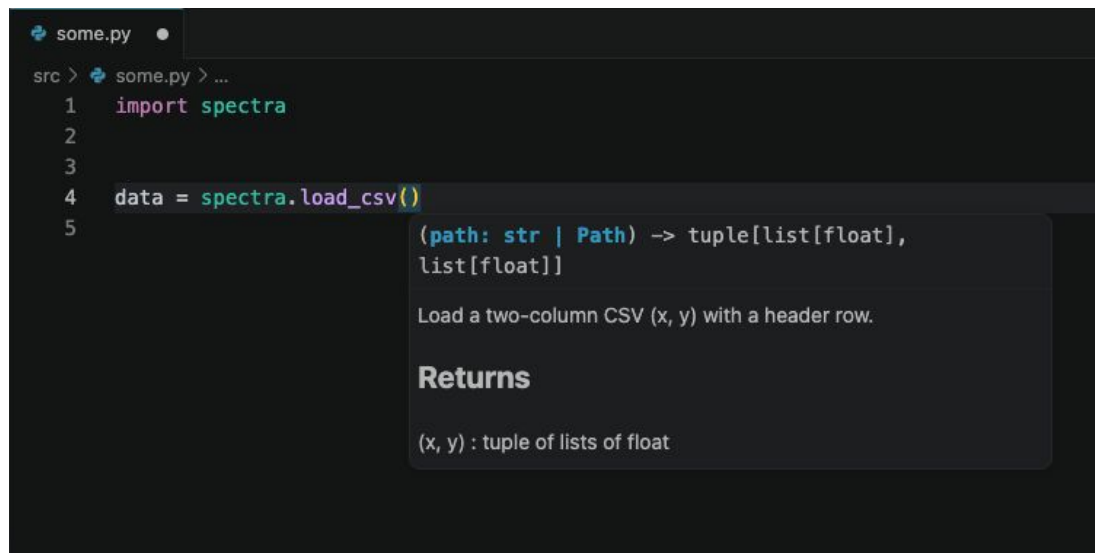
typehinty:

čo je x? čo je y? čo funkcia vracia?

docstring:

ako sa funkcia správa

Vývojarská Empatia



```
some.py
src > some.py > ...
1  import spectra
2
3
4  data = spectra.load_csv()
5
```

(path: str | Path) -> tuple[list[float], list[float]]

Load a two-column CSV (x, y) with a header row.

Returns

(x, y) : tuple of lists of float

Cvičenie 4 | Pridaj Empatiu, verzia 0.2.0

(5 min.)

- Vyberte si jednu funkciu
 - pridajte docstring a typehinty
- Vybuildujte to znovu (čo na Vás nakričal build?)
- Fixnite to a skuste buildnuť znovu
- Nainštalujte si verziu 0.2.0 v projekte z cvičenia 3, cítite zlepšenie?

```
pip install build  
python -m build
```

Upload na PyPI

- vytvorenie účtu na Test PyPI indexe: <https://test.pypi.org/>
- potvrdenie emailu, 2F-Autentifikácia
- account settings -> API Tokens -> Add API Token

Upload na PyPI

```
(venv.) majirky@pc-adam spacelab_ss26 % pip install twine  
(venv.) majirky@pc-adam spacelab_ss26 % twine check dist/* # kontrola pred uploadom  
(venv.) majirky@pc-adam spacelab_ss26 % twine upload --repository testpypi dist/*
```

.....

View at:

<https://test.pypi.org/project/<package-name>/<version>/>

.....

```
(venv.) majirky@pc-adam spacelab_ss26 % pip install --index-url https://test.pypi.org/simple/  
<package-name>
```

Next Steps

Hrať sa s `__init__.py`

- Dovoľuje jednoduchšie importy funkcií

UV

- Rýchly a moderný Python tooling, <https://docs.astral.sh/uv/>

Pozrieť sa na lintery a formátovače pre Python

- Nastaviť ich v `pyproject.toml`,
- Automatizovať kontrolu kvality a formátovanie kódu v projekte

Prax, prax, prax

- Research repo: <https://github.com/wongkinyiu/yolov9>
- Nadizajnovat' funkcie pre tréning a inferenciu modelu
- Zabalit' a publikovat'



Otázky?

amajirsky@gmail.com